

07 • Transformer Architecture Refresher

Source: [fanout.sh / inference-eng / 00-06](https://fanout.sh/inference-eng/00-06) • ~9 min

IN SHORT

A transformer is an **embedding lookup**, a stack of **identical blocks** (attention + MLP, each adding onto a running vector), and a **head** that scores the whole vocabulary. Seven numbers in Llama 3 8B's ~30-line config decide its whole bill: **8.03B parameters** (70% in the MLPs), **~15B operations per token**, and a KV cache of **128 KB per token**. New models mostly swap parts of this skeleton, and each swap moves one line of the bill.

Llama 3 8B config: seven numbers, three bills

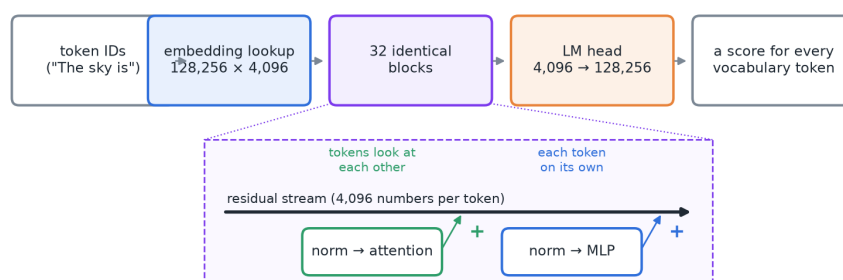
		memory	ops / token	KV cache growth
hidden_size	4,096	●	●	●
intermediate_size	14,336	●	●	○
num_attention_heads	32	○	○	●
num_hidden_layers	32	●	●	●
num_key_value_heads	8	●	●	●
vocab_size	128,256	●	●	○
torch_dtype	bfloat16	●	○	●

attention heads matter through head size ($4,096 \div 32 = 128$); dtype sets bytes, not op count

The seven config numbers that matter; and which bill each one feeds.

1. The whole machine

The whole machine: lookup, 32 blocks that add onto a running vector, then score the vocabulary



Token IDs pick rows of the embedding table; 32 blocks add their results onto the residual stream; the LM head scores all 128,256 tokens.

- Each token ID picks a **row of the embedding table**: a vector of **4,096 numbers**.
- The vectors pass through **32 identical blocks**. Each block has **attention** (tokens look at each other) and an **MLP** (each token processed on its own).

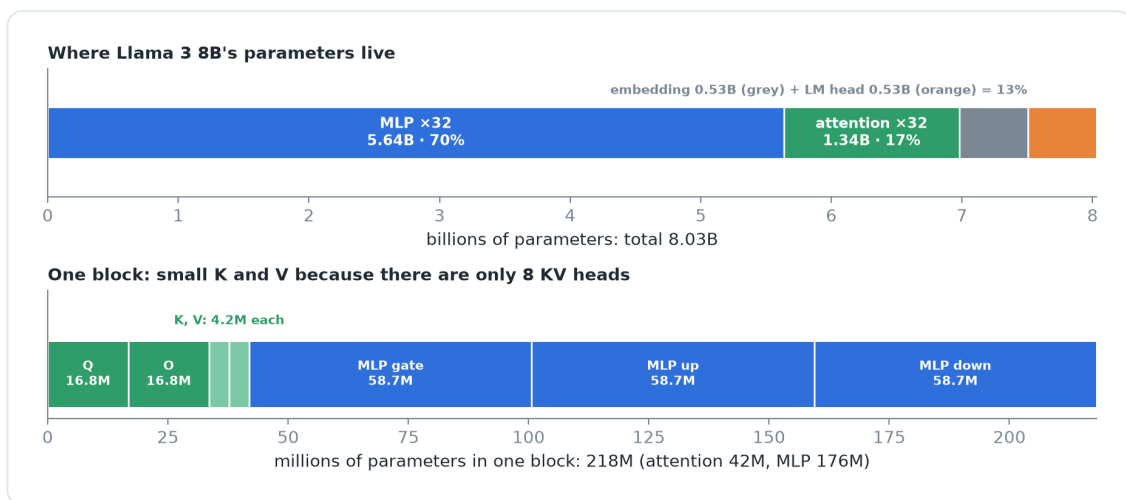
- Each part reads a **normalized copy** of the vector and **adds** its result back: that running vector is the **residual stream**.
- The **LM head** turns each final vector into a score for every vocabulary token (**128,256**).

KEY IDEA

That's the entire architecture. Everything an inference engine does is about running this stack faster.

2. Counting parameters from the config

Piece	Shape	Parameters
Embedding table	$128,256 \times 4,096$	~525M
Attention Q, O (per block)	$4,096 \times 4,096$ each	~17M each
Attention K, V (per block)	$4,096 \times 1,024$ each (8 KV heads $\times$ 128)	~4M each
Attention total (per block)		~42M
MLP (per block)	3 matrices of $4,096 \times 14,336$	~176M
One block		~218M $\rightarrow \times 32 \approx 6.98B$
LM head (separate, not tied)	$4,096 \times 128,256$	~525M
Total		8.03B (the "8B")



MLPs hold 70% of the weights, attention 17%, embedding + head 13%. Within a block, K and V are small because there are only 8 KV heads.

3. Parameters $\rightarrow$ bytes

Bytes = parameters $\times$ bytes per parameter (see 04): **16 GB** at 16-bit, **~8 GB** at 8-bit, **~4 GB** at 4-bit (plus a little for scaling factors). Decode reads **every byte every step**, so precision is both a **memory knob** and a **speed knob**.

INTUITION

The MLPs' 70% is where quantization saves most of the bytes, and most of the time.

4. Parameters → operations

Each weight is used once per token as **one multiply + one add**, so a token costs **~2 operations per parameter**, except the **embedding table**, which is a lookup (free). The LM head is a real matmul and one of the biggest matrices.

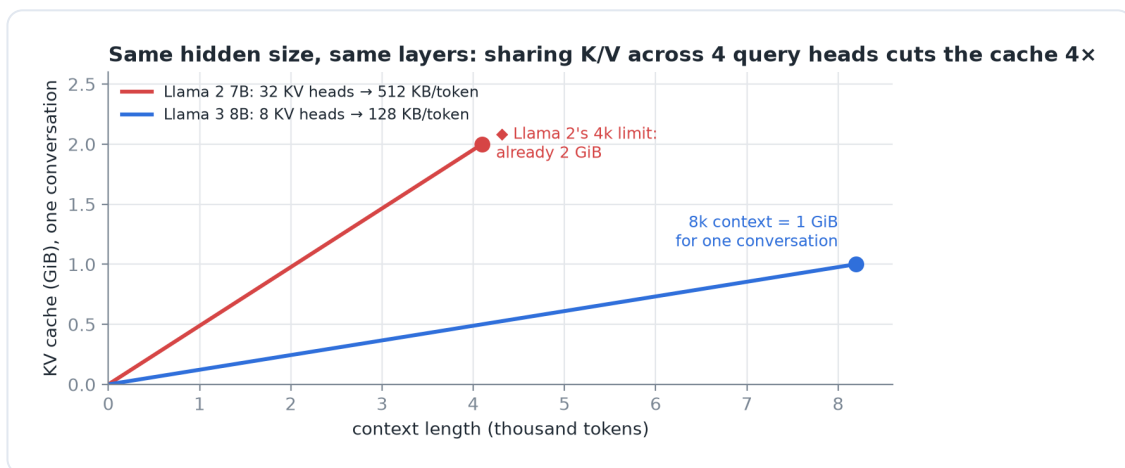
$$2 \times 7.5\text{B non-embedding params} \approx 15\text{B ops per token}$$

The same 15B whether the token is in the prompt or generated; what changes is **how they arrive**:

- **Prefill**: 500 prompt tokens together = **7.5 trillion ops** in one batch, keeping the math units busy.
- **Decode**: 15B ops against 16 GB of weight reads $\approx$ **1 op per byte**: the GPU mostly waits on memory (see 01, 04).

5. The KV cache: the line that grows

Every block keeps the **keys and values of every token it has seen**, so they're never recomputed. From the config: 8 KV heads $\times$ 128 $\times$ 2 (K, V) $\times$ 2 bytes = **4 KB per token per layer**, $\times$ 32 layers = **128 KB per token**. A full **8k context $\approx$ 1 GB**, for one conversation.



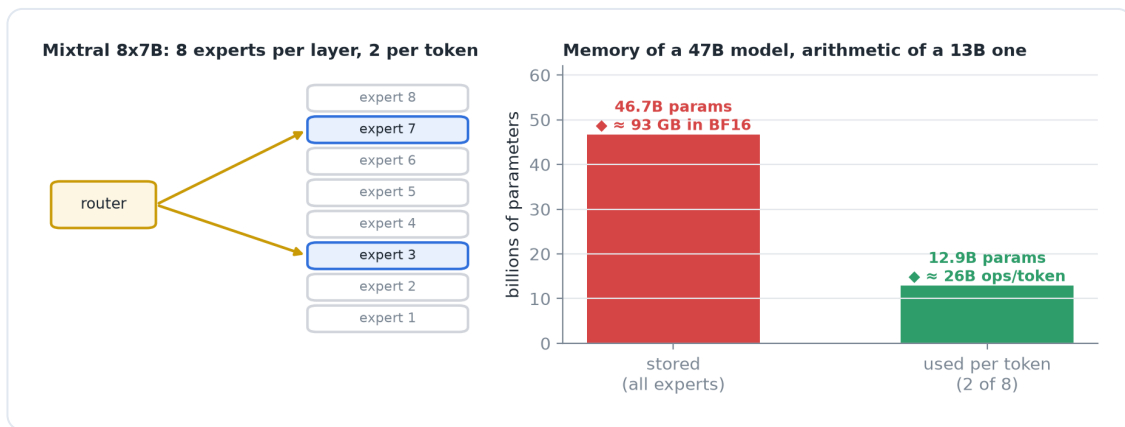
Llama 2 7B gave each of its 32 query heads its own K and V: 512 KB per token, 4× Llama 3 8B, with the same hidden size and layers.

KEY IDEA

Sharing keys and values across groups of query heads (GQA) is one of the cheapest wins in modern architectures.

6. Variants on the same skeleton

- **Mixture of experts (MoE)**: replace the one big MLP with many smaller **experts**; a **router** picks a few per token.



Mixtral 8x7B: 8 experts per layer, 2 per token. 46.7B parameters stored, 12.9B used per token: memory and compute come apart.

- **Attention variants:** fewer KV heads or compressed keys and values **shrink the cache line** directly.
- **Positions:** Llama rotates each query and key by a **position-dependent angle (RoPE)**. With no position table to outgrow, this is part of how **Llama 3.1 reached 128k tokens** of context.

Same skeleton, swapped parts: each swap moves one line of the bill

	weight memory	ops per token	KV cache	max context
mixture of experts	↑ all experts	↓ only active	*	*
fewer KV heads / compressed K,V	*	*	↓ shrinks directly	*
rotary positions (RoPE)	*	*	*	↑ no position table (Llama 3.1: 128k)
lower precision (4/16 vs 16 bits)	↓ bytes	*	*	*

Each swap shows up on exactly one line of the bill.

7. Three corrections

Three corrections (top: the myth, bottom: what the config says)

"8B params = 16 GB needed"	"every param costs compute"	"attention holds the weights"
weights are the floor: + KV cache + working memory; at scale the cache can exceed the model	embedding = a lookup (0 ops); in MoE most experts sit idle per token	MLP 176M vs attention 42M per block: > 4× more in the MLP

Parameter count isn't memory needed; not every parameter costs compute; the MLPs, not attention, hold most weights.

8. Recap

Lesson 07 on one card

SHAPE

embedding lookup → 32 blocks
(norm → attention, norm → MLP,
added to the residual stream) → head

PARAMETERS

8.03B from the config
MLP 70% · attention 17%
embedding + head 13%

OPS PER TOKEN

$\approx 2 \times$ non-embedding params
 $\approx 2 \times 7.5\text{B} = 15\text{B}$
same for prompt or generated

CACHE

128 KB per token (8 KV heads)
8k context = 1 GB
Llama 2 7B: 512 KB

Interview cheat sheet

Revise from these pages alone. ♦ = my addition beyond the video; everything else is from the lesson. Builds on 01–06.

Say it in 30 seconds

A decoder transformer is an embedding lookup, a stack of identical blocks with attention and an MLP adding onto a residual stream, and an LM head over the vocabulary. From the config you can count parameters (Llama 3 8B: 8.03B, 70% in MLPs), turn them into bytes with the precision, and estimate ~ 2 ops per non-embedding parameter per token. The KV cache, set by the number of KV heads, is the line that grows. MoE, GQA and RoPE each move one line of that bill.

Only numbers worth memorizing

- **Llama 3 8B config:** hidden 4,096 · MLP 14,336 · 32 layers · 32 heads · 8 KV heads · vocab 128,256 · bf16.
- **Shares:** MLP 70%, attention 17%, embedding + head 13%. **Mixtral 8x7B:** 46.7B stored, 12.9B active.

Derive, don't memorize

Let h = hidden, i = MLP size, V = vocab, L = layers, $H_{kv} \cdot d$ = KV width ($8 \times 128 = 1,024$).

1. Parameters from the config

$$\text{block} = 2h^2 + 2h \cdot H_{kv}d + 3hi \quad N = L \cdot \text{block} + 2Vh = 32 \times 218\text{M} + 2 \times 525\text{M} \approx 8.03\text{B}$$

The three terms are Q and O ($h \times h$), K and V ($h \times H_{kv}d$), and the three MLP matrices ($h \times i$).

INTUITION

The $2Vh$ assumes an untied LM head (Llama 3 8B). If embeddings are tied ♦, count Vh once.

2. Ops per token = 2 × non-embedding parameters

$$2 \times (8.03\text{B} - 0.53\text{B}) \approx 15\text{B ops/token}$$

$$\text{prefill 500 tokens} = 7.5\text{T ops per weight read} \quad \text{decode} \approx \frac{15\text{B ops}}{16\text{ GB}} \approx 1\text{ op/byte}$$

3. MoE splits memory from compute

$$\text{memory} \propto N_{\text{total}} = 46.7\text{B} \quad \text{ops/token} \approx 2N_{\text{active}} = 2 \times 12.9\text{B}$$

INTUITION

♦ In BF16 that's ~ 93 GB to host but only $\sim 26\text{B}$ ops per token. KV per token comes straight from the config too: see 05, derivation 2.

Swap → which line of the bill

Swap	Weight memory	Ops per token	KV cache	Context
Mixture of experts	↑ all experts	↓ only active		
Fewer KV heads / compressed K,V			↓	
RoPE positions				↑ (Llama 3.1: 128k)
Lower precision	↓ bytes			

Rapid-fire Q&A

Question	Crisp answer
Where do most parameters live?	MLPs (~70%); > 4× attention per block.
Does 8B params mean 16 GB is enough?	No: weights are the floor; KV cache and working memory add on top.
Which parameters cost no compute?	The embedding lookup, and idle experts in MoE.
Why did Llama 3 use 8 KV heads?	GQA cuts the cache 4× vs Llama 2 7B (128 vs 512 KB/token).
Why quantize MLPs first?	They hold most bytes, and decode time $\propto$ bytes read.

TRAP

- Equating parameter count with required memory.
- Assuming MoE is cheap to host because it's cheap per token.